

1. Introdução

Este teste tem como objetivo avaliar suas habilidades técnicas e organizacionais na construção de um sistema web completo. O foco será na implementação de uma aplicação funcional, bem estruturada e seguindo boas práticas de desenvolvimento.

2. Critérios de avaliação

Durante a correção, os seguintes critérios serão avaliados:

- Conhecimento em back-end e front-end.
- Organização e estruturação do projeto.
- Análise e entendimento dos requisitos.
- Qualidade e clareza do código.
- Conhecimento em banco de dados e APIs.
- Lógica de programação.
- Controle de versão utilizando Git.

3. Descrição do problema

Uma academia precisa de um sistema web para acompanhar a evolução dos seus alunos em relação ao Índice de Massa Corporal (IMC). Para isso, a academia deseja desenvolver uma aplicação onde os alunos possam ser cadastrados, assim como os professores que realizarão suas avaliações.

O sistema deverá permitir que as avaliações sejam armazenadas e consultadas, possibilitando a análise da evolução dos alunos.

O acesso ao sistema será feito por três tipos de usuários:

- Administrador
- Professor
- Aluno

4. Requisitos

4.1 Requisitos funcionais

- O sistema deve permitir o cadastro de usuários com os perfis: administrador, aluno ou professor.
- Deve ser possível ativar ou inativar um usuário.
- O sistema deve permitir o cadastro de avaliações de IMC dos alunos, calculadas com base na altura e peso.

- Deve gravar a classificação do IMC calculado com base na seguinte tabela:

IMC	Classificação
Menor que 18.5	Abaixo do peso
18.5 - 24.9	Peso normal

IMC	Classificação
25 - 29.9	Sobrepeso
30 - 34.9	Obesidade grau I
35 - 39.9	Obesidade grau II
Acima de 40	Obesidade grau III

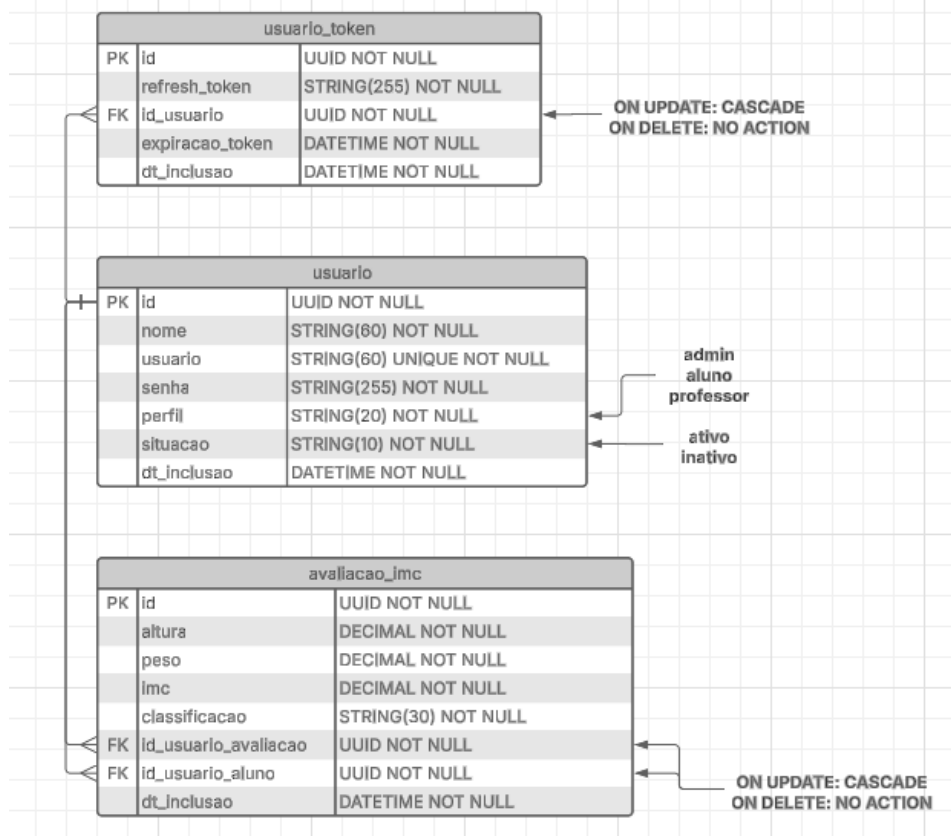
- Deve haver uma consulta das avaliações de IMC com filtro por aluno ou professor.
- Deve ter autenticação com usuário e senha e pode ser acessado pelos administradores, professores ou alunos.

4.2 Regras de negócio

- Usuário administrador:
 - Pode cadastrar, editar e excluir (se não houver avaliações vinculadas) usuários de qualquer perfil.
 - Pode cadastrar, editar e excluir avaliações de IMC.
 - Pode consultar avaliações de qualquer aluno.
- Usuário professor:
 - Pode cadastrar e editar alunos.
 - Pode cadastrar e editar avaliações de IMC.
 - Pode consultar avaliações de seus alunos.
- Usuário aluno:
 - Pode consultar apenas suas próprias avaliações.
- Usuários inativos:
 - Não podem acessar o sistema.
 - Não podem ter novas avaliações cadastradas.

5. Modelo de dados (DER)

Entendemos que o diagrama abaixo implementa o necessário para armazenar os dados para implementar a solução, mas fique a vontade caso precise modificá-lo.



6. Tecnologias e ferramentas que devem ser utilizadas no projeto

Crie um repositório monorepo no GitHub e faça o versionamento do código seguindo o padrão de **Conventional Commits**.

6.1 Back-end

- Typescript.
- Express.js – Framework para aplicações Node.js.
- Validação da entrada de dados nos endpoints com celebrate, yup ou zod.
- TypeORM – ORM para trabalhar com bancos de dados. Utilizar migrations para versionamento do banco.
- SQLite ou SQL Server como banco de dados.
- Fluxo de autenticação com JWT (JSON Web Token).

6.2 Front-end

- Typescript.
- React – Biblioteca JavaScript para construção de interfaces.
- Next.js – Framework React para aplicações com renderização híbrida (SSR/SSG).
- Chakra UI – Biblioteca de componentes UI para React.
- React Query – Gerenciamento de dados assíncronos e cache.
- React Hook Form – Biblioteca de formulários para React. Validar schemas com yup ou zod.
- Axios - Biblioteca para fazer a requisição dos endpoints do backend.

7. Entrega

Ao finalizar o teste envie um e-mail para **dev@sooro.com.br** com o link do repositório GitHub da sua implementação.

Prazo para entrega: 7 dias.